OS Lab

Skill-11

To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_HISTORY 10
#define MAX_COMMANDS 10
#define MAX_INPUT 1024

// ==================== HISTORY ====================

char *history[MAX_HISTORY];
int history_count = 0;

// Store command in history
void add_history(char *command) {
   // If history is full, remove the oldest command
   if (history_count == MAX_HISTORY) {
      free(history[0]);

      for (int i = 1; i < MAX_HISTORY; i++) {
         history[i - 1] = history[i];
      }

      history_count--;
   }

   history[history_count] = strdup(command);
   history_count++;
}

// Display history
```

```
void show_history() {
   for (int i = 0; i < history_count; i++) {
      printf("%d  %s\n", i + 1, history[i]);
   }
}

// Free history memory
void free_history() {
   for (int i = 0; i < history_count; i++) {
      free(history[i]);
   }
}

// ==================== PIPELINE ====================

// Execute pipeline commands
void execute_pipeline(char *input) {

   char *commands[MAX_COMMANDS];
   int command_count = 0;

   // Split input using |
   char *command = strtok(input, "|");

   while (command != NULL && command_count < MAX_COMMANDS) {
      commands[command_count++] = command;
      command = strtok(NULL, "|");
   }

   int pipes[MAX_COMMANDS - 1][2];
   pid_t processes[MAX_COMMANDS];

   // Create pipes
   for (int i = 0; i < command_count - 1; i++) {
      if (pipe(pipes[i]) == -1) {
         perror("pipe");
         return;
      }
   }

   // Create processes
   for (int i = 0; i < command_count; i++) {

      processes[i] = fork();
```

```
if (processes[i] == -1) {
   perror("fork");
   return;
}

if (processes[i] == 0) {

   // Connect input from previous pipe
   if (i > 0) {
      dup2(pipes[i - 1][0], STDIN_FILENO);
   }

   // Connect output to next pipe
   if (i < command_count - 1) {
      dup2(pipes[i][1], STDOUT_FILENO);
   }

   // Close all pipe file descriptors
   for (int j = 0; j < command_count - 1; j++) {
      close(pipes[j][0]);
      close(pipes[j][1]);
   }

   // Convert command into arguments
   char *args[MAX_COMMANDS];
   int arg_count = 0;

   char *token = strtok(commands[i], " \t\n");

   while (token != NULL && arg_count < MAX_COMMANDS - 1) {
      args[arg_count++] = token;
      token = strtok(NULL, " \t\n");
   }

   args[arg_count] = NULL;

   // Execute command
   if (arg_count > 0) {
      execvp(args[0], args);
      perror("execvp");
   }

   exit(EXIT_FAILURE);
```

```
        }
    }

    // Parent closes all pipes
    for (int i = 0; i < command_count - 1; i++) {
        close(pipes[i][0]);
        close(pipes[i][1]);
    }

    // Wait for all processes
    for (int i = 0; i < command_count; i++) {
        waitpid(processes[i], NULL, 0);
    }
}

// ==================== MAIN SHELL ====================

int main() {

    char input[MAX_INPUT];

    while (1) {

        printf("myshell$ ");
        fflush(stdout);

        // Read command
        if (fgets(input, sizeof(input), stdin) == NULL) {
            break;
        }

        // Remove newline
        input[strcspn(input, "\n")] = '\0';

        // Ignore empty command
        if (strlen(input) == 0) {
            continue;
        }

        // Exit
        if (strcmp(input, "exit") == 0) {
            break;
        }
```

```
    // History command
    if (strcmp(input, "history") == 0) {
        show_history();
        continue;
    }

    // Store command
    add_history(input);

    // Check whether command contains pipe
    if (strchr(input, '|') != NULL) {

        execute_pipeline(input);

    } else {

        // Normal command
        char *args[MAX_COMMANDS];
        int arg_count = 0;

        char *token = strtok(input, " \t\n");

        while (token != NULL && arg_count < MAX_COMMANDS - 1) {
            args[arg_count++] = token;
            token = strtok(NULL, " \t\n");
        }

        args[arg_count] = NULL;

        if (arg_count > 0) {

            pid_t pid = fork();

            if (pid == 0) {
                execvp(args[0], args);
                perror("execvp");
                exit(EXIT_FAILURE);
            }

            waitpid(pid, NULL, 0);
        }
    }
}
```

```
    free_history();

    printf("Shell terminated.\n");

    return 0;
}
```

